



COMPOSITE PATTERN

1 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành, sinh viên có thể:

- Hiểu rõ Composite Pattern: khái niệm, cấu trúc (Component – Leaf – Composite) và phân biệt với Decorator.
- Nhận diện và áp dụng khi hệ thống có cấu trúc dạng cây.
- Thiết kế và triển khai mô hình Order phân cấp bằng C# theo Composite.
- Biết vẽ UML (trước/sau) và xử lý đệ quy để tính toán (ví dụ: tổng giá trị Order).
- Phát triển tư duy thiết kế cho các bài toán dữ liệu phân cấp như menu, category tree, file system, BOM.

2 VẤN ĐỀ (PROBLEM)

Giả sử hệ thống có: Order, OrderItem, ComboProduct (gồm nhiều Product), Bundle khuyến mãi

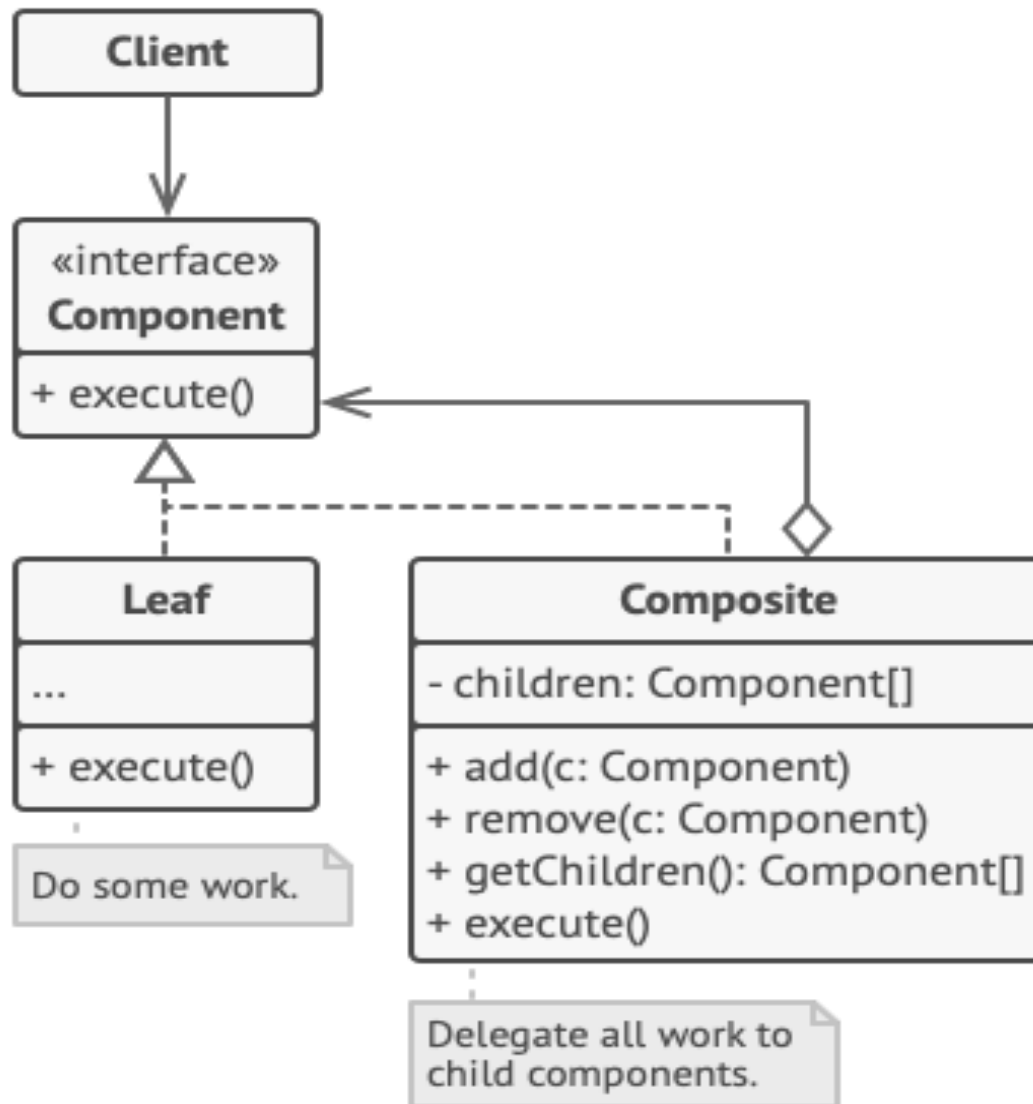
✗ Thiết kế cũ (if-else phân cấp)

```
public class Order
{
    public decimal CalculateTotal()
    {
        decimal total = 0;
        foreach (var item in Items)
        {
            if (item is Product)
                total += item.Price;
            else if (item is ComboProduct)
                total += item.GetComboPrice();
        }
        return total;
    }
}
```

🚩 Vấn đề: if-else phình to, Không mở rộng, Vi phạm OCP

3

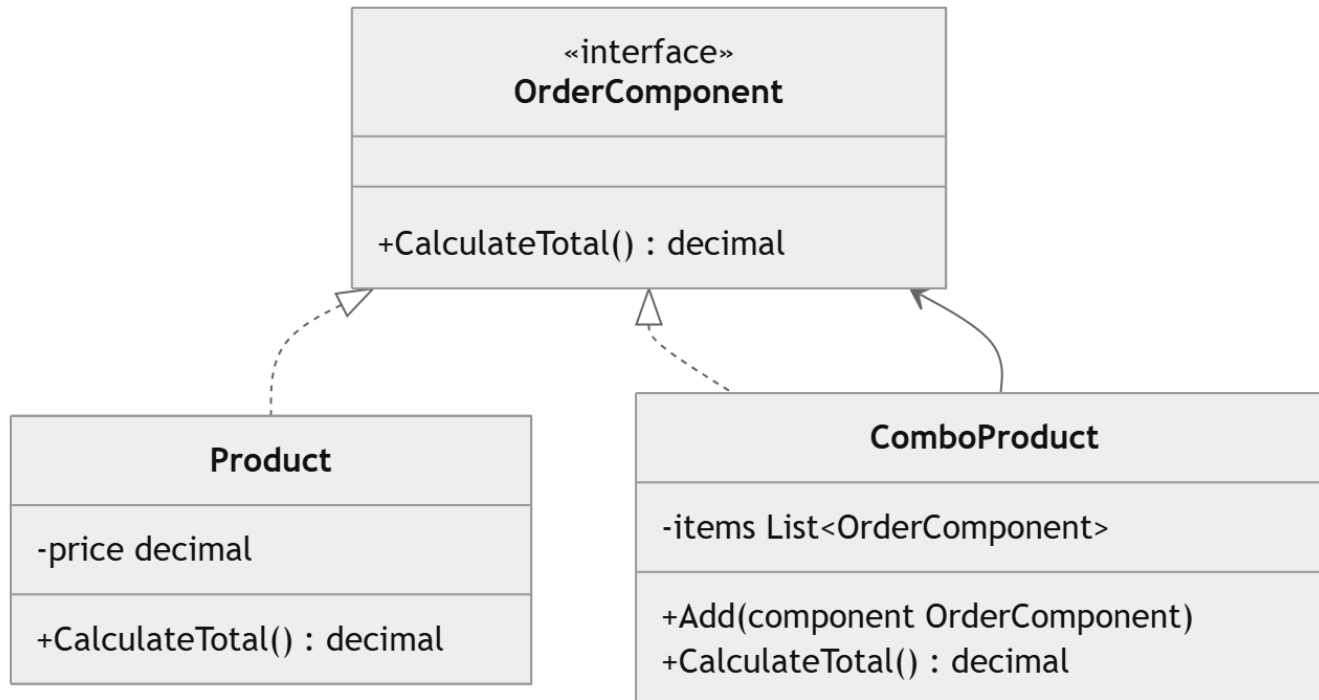
Structure of Composite Pattern



3 Composite pattern: Case Study

📌 Định nghĩa

Composite Pattern cho phép bạn tổ chức các đối tượng thành cấu trúc cây để biểu diễn quan hệ “whole–part” (tổng thể – thành phần), đồng thời cho phép xử lý các đối tượng đơn lẻ (Leaf) và tổ hợp (Composite) theo cùng một cách.



5 TRIỂN KHAI COMPOSITE (Step 1 & 2)

Step 1 – Component

```
public abstract class OrderComponent
{
    public abstract decimal CalculateTotal();
}
```

Step 2 – Leaf (Product)

```
public class Product : OrderComponent
{
    public decimal Price { get; set; }
    public Product(decimal price) { Price = price; }
    public override decimal CalculateTotal()
    {
        return Price;
    }
}
```

5 TRIỂN KHAI COMPOSITE (Step 3 & 4)

Step 3 – Composite (ComboProduct)

```
public class ComboProduct : OrderComponent
{
    private readonly List<OrderComponent> _items = new();
    public void Add(OrderComponent component) {
        _items.Add(component); }
    public override decimal CalculateTotal()
    {
        decimal total = 0;
        foreach (var item in _items) { total +=
item.CalculateTotal(); }
        return total;
    }
}
```

Step 4 – Sử dụng

```
var combo = new ComboProduct();
combo.Add(new Product(100)); combo.Add(new Product(200));
Console.WriteLine(combo.CalculateTotal()); // 300
```

6 & **8** SO SÁNH VÀ PHÂN BIỆT

BEFORE	AFTER
if-else type checking	Polymorphism
Khó mở rộng	Mở rộng dễ
Không hỗ trợ nested	Hỗ trợ vô hạn cấp
Vi phạm OCP	Tuân thủ OCP

Composite	Decorator
Quan hệ part-whole	Quan hệ wrap
Tạo cấu trúc cây	Tăng hành vi

7 ỨNG DỤNG TRONG ORDER MANAGEMENT

Composite phù hợp cho:

Order → nhiều OrderItem | ComboProduct → nhiều Product
Category Tree | Discount Rule Tree | Approval Workflow Tree

Ví dụ thực tế:

Microsoft Windows File System (Folder chứa Folder/File)

Oracle Java Swing Component tree

Apache Software Foundation Maven dependency tree

9 BÀI TẬP THỰC HÀNH

 Bài 1 – Refactor Order: if-else, UML BEFORE/AFTER, Test nested combo 3 cấp.

 Bài 2 – Discount Tree: Discount sản phẩm/combo/toàn bộ order theo cấu trúc cây.

 Bài 3 – Approval Workflow: Manager, Director, CEO phê duyệt hoặc chuyển tiếp.

10 THẢO LUẬN & KẾT LUẬN

THẢO LUẬN LỚP:

Tăng số class? Khi nên dùng thay List thuần? Hợp DB quan hệ?

KẾT LUẬN

- ✓ Thiết kế cấu trúc phân cấp, Loại bỏ if-else kiểm tra type
- ✓ Hỗ trợ đệ quy tự nhiên, Mở rộng dễ dàng, Tuân thủ OCP